

Section 1: Preliminary Analysis

Data Structure Observations

Each training file contains a 5-minute wrist accelerometer recording summarized as 300 rows (one row per second), with six columns: mean_x, mean_y, mean_z, std_x, std_y, std_z. Each file belongs to a specific user folder (60 training users) and carries a single activity label for the entire recording. The task is therefore file-level multi-class classification (6 classes) rather than frame-level segmentation. The dataset comprises 11,020 training files and 6,849 test files.

Examining the raw data revealed several structural properties that directly informed method design:

Fixed-length sequences: Every file has exactly 300 rows (mean = 300.0, std = 0.0, min = 300.0, max = 300.0 as confirmed by the shape summary), meaning temporal modeling can treat each recording as a uniform-length time series of length 300. This makes both CNN-based sequence models and global statistical feature extractors straightforward to apply without variable-length padding complications.

Train files: 11020 Test files: 6849
Train shape per file:

	n_rows
count	11020.0
mean	300.0
std	0.0
min	300.0
25%	300.0
50%	300.0
75%	300.0
max	300.0
dtype: float64	

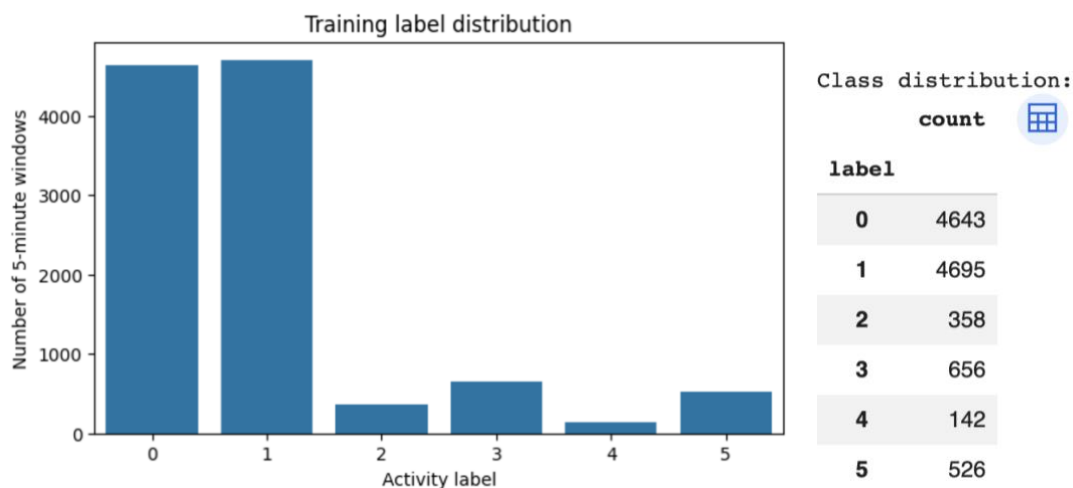
Test shape per file:

	n_rows
count	6849.0
mean	300.0
std	0.0
min	300.0
25%	300.0
50%	300.0
75%	300.0
max	300.0

Pre-summarized signal: The raw sensor readings are already aggregated into per-second mean and standard deviation. This means the input is not raw accelerometer samples at,

say, 50 Hz, but rather one-second summary statistics. Two important implications follow: (1) high-frequency FFT features computed on the 300-row sequence correspond to cycles over 300-second windows, not the original sensor frequency; (2) very short temporal patterns (sub-second motion bursts) are not recoverable from this representation.

Strong class imbalance: The training label distribution (see chart below) reveals severe imbalance across the 6 activity classes. Classes 0 and 1 each account for roughly 4,600+ samples and together dominate the training set, while classes 2, 3, 4, and 5 are all substantially underrepresented. The exact training counts are: class 0 — 4,643, class 1 — 4,695, class 2 — 358, class 3 — 656, class 4 — 142, class 5 — 526. Class 4 is the rarest with fewer than 150 samples — roughly 30× fewer than classes 0 and 1.



This imbalance makes macro-F1 the appropriate metric rather than accuracy: a model that predicts only class 0 or 1 would achieve high accuracy but near-zero macro-F1. The minority classes (especially class 2 with 358 samples and class 4 with 142) become the dominant driver of score differences across methods, as even small absolute changes in their per-class F1 scores heavily move the macro average.

Waveform characteristics by class: Examining example 5-minute sequences for each label reveals qualitatively distinct temporal patterns across the six activity classes.



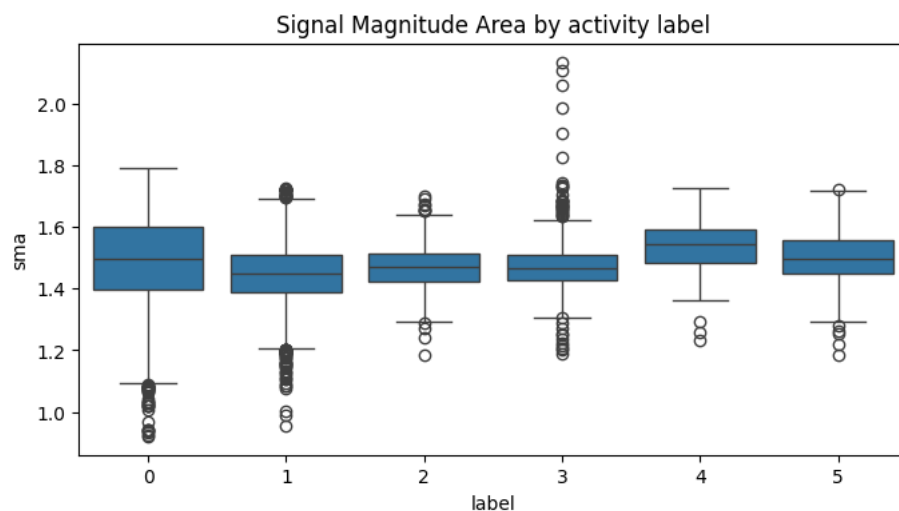
Label 0 shows near-flat, low-amplitude signals suggesting rest or a still posture. Labels 1 and 2 show moderate oscillation with occasional bursts. Label 3 exhibits high-amplitude, irregular multi-axis variation. Label 4 shows a mixed pattern with a transition partway through the recording. Label 5 has relatively stable mean values with a sharp step change visible around second 150 – 200, suggesting a discrete posture or orientation shift.

These waveforms confirm that the mean x/y/z channels encode both activity intensity and wrist orientation, and that temporal dynamics — including transitions, periodicity, and non-stationarity — vary substantially across classes.

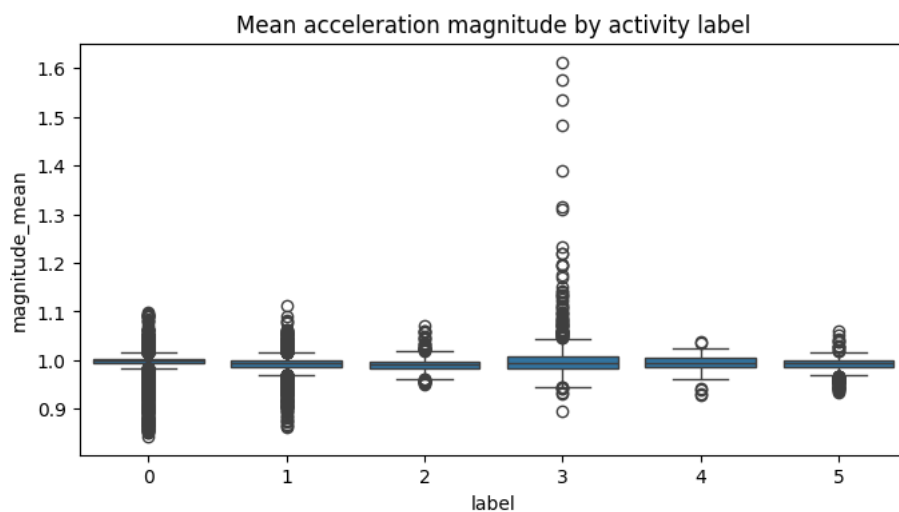
Signal Magnitude Area by class: The SMA (signal magnitude area, computed as the sum of absolute axis means per second) shows overlapping distributions across classes, with

class 0 having the widest spread (std = 0.150033) and class 2 the narrowest (std = 0.077149). Notably, class 3 has the most high-end outliers in SMA (values reaching above 2.0), consistent with the high-amplitude waveform observed in its time series.

The x-axis shows activity labels 0 – 5. All classes have median SMA near 1.4 – 1.5g. Class 0 has the largest interquartile range and many low outliers below 1.0. Class 3 has several extreme outliers above 1.8 – 2.2. Classes 2, 4, and 5 have tighter boxes. Overall, SMA alone does not cleanly separate classes, motivating richer feature engineering.



Mean acceleration magnitude by class: The mean acceleration magnitude (Euclidean norm of mean_x/y/z per file) is similarly uninformative on its own. All classes cluster near 1.0g (consistent with gravity being the dominant component in wrist accelerometry), with class 3 again showing the most outlier spread.



All six classes show median values tightly clustered around 1.0g. Class 3 has multiple extreme outliers above 1.3 – 1.6g. Classes 0 and 1 have tightly concentrated distributions with a few low-value outliers. Classes 2, 4, and 5 are compact. The plot illustrates that magnitude alone is not discriminative, reinforcing the need for temporal and distributional features.

The similarity of SMA and magnitude means across all six classes (all within ~0.05g of each other) confirms that the classification problem cannot be solved with simple intensity features alone. This directly motivated the development of richer temporal, frequency-domain, and distributional features in subsequent versions.

	sma		magnitude_mean	
	mean	std	mean	std
label				
0	1.485381	0.150033	0.995992	0.022489
1	1.447392	0.096836	0.991060	0.016821
2	1.470360	0.077149	0.992319	0.017354
3	1.474770	0.092160	1.005600	0.062055
4	1.537327	0.087320	0.993842	0.017460
5	1.503201	0.085656	0.991313	0.015139

Missing values: No missing values were found in either the training or test sets, so no imputation was required at any stage.

Missing values in train: 0
Missing values in test: 0

User-level variation: The 60 training users show substantial between-user variation in their sensor baselines. Wrist placement angle, wrist orientation, and individual motion style all shift the absolute values of mean_x/y/z. This was confirmed by the GroupKFold OOF drop: when validation folds contain entirely unseen users, OOF F1-macro fell from ~0.74 (StratifiedKFold) to ~0.71 (GroupKFold), a gap of ~0.03 attributable purely to user distribution shift.

Naive Baseline Performance

submission-2 (RF-only baseline, 0.7209): This was the first submission, labeled "test" on Kaggle. It was a pure Random Forest with no deep learning component, trained on 30 features per file: global mean, std, max, and min for each of the 6 axes, plus per-axis rolling-window means (window size 10 rows), and resultant magnitude mean/std. No cross-validation, no early stopping, no frequency-domain features. The rolling-window mean was the only temporal feature — it computed the average of a 10-second sliding window and then averaged that again over the full file, collapsing any temporal dynamics into a single scalar per axis.

submission-3 (RF + CNN-LSTM hybrid, 0.7537): The first hybrid ensemble, combining the same RF (on the same 26-column global-stat feature set, now without the rolling mean) with a CNN-LSTM deep learning model. The CNN-LSTM reshaped the 300-row sequence into 10 windows of 30 seconds each, processed each window with a TimeDistributed Conv1D(64, kernel=3) and GlobalAveragePooling, then passed the resulting 10-step sequence to an LSTM(64, dropout=0.2). Final predictions were a 50/50 probability average of RF and CNN-LSTM outputs. The model was trained for a fixed 20 epochs with batch size 16, no early stopping. The 0.7537 score was a substantial jump (+0.0328) over submission-2 purely from adding the CNN-LSTM temporal model.

Baseline deductions: The 0.7209 RF-only result confirms that global statistics alone are not sufficient: the macro-F1 ceiling for flat feature methods is limited by minority classes (especially 2 and 5), which require richer temporal and frequency-domain representations. The +0.0328 jump from adding CNN-LSTM over the RF alone demonstrates that the sequential 300-step structure contains discriminative temporal information that global statistics discard. This motivated systematic development of both richer hand-crafted features (FFT, autocorrelation, segment statistics) and improved deep learning architectures in subsequent versions.

Class Imbalance and Per-Class Difficulty

Two OOF classification reports were produced in the private experiments phase and together reveal the per-class difficulty across all method variants.

The GroupKFold LGB OOF report (macro-F1: 0.7123) and the user-median LGB OOF report (macro-F1: 0.7206) give slightly different per-class views due to the normalization applied in the latter.

00F GroupKFold LightGBM macro-F1: 0.7123
 Fold scores: [0.6521, 0.7409, 0.7251, 0.6843, 0.7024]

	precision	recall	f1-score	support
0	0.9544	0.9698	0.9621	4643
1	0.8817	0.9188	0.8999	4695
2	0.3583	0.1201	0.1799	358
3	0.6486	0.7287	0.6863	656
4	0.8841	0.8592	0.8714	142
5	0.7657	0.6027	0.6745	526
accuracy			0.8872	11020
macro avg	0.7488	0.6999	0.7123	11020
weighted avg	0.8759	0.8872	0.8789	11020

Fold 5 user-median LGB macro-F1: 0.7077
 00F user-median LGB macro-F1: 0.7206

	precision	recall	f1-score	support
0	0.9516	0.9696	0.9605	4643
1	0.8831	0.9088	0.8958	4695
2	0.3642	0.1648	0.2269	358
3	0.6404	0.7683	0.6985	656
4	0.8978	0.8662	0.8817	142
5	0.7978	0.5627	0.6600	526
accuracy			0.8848	11020
macro avg	0.7558	0.7068	0.7206	11020
weighted avg	0.8768	0.8848	0.8781	11020

The per-class difficulty pattern is consistent across both reports:

- **Classes 0 and 1** (~4,643 and ~4,695 samples each): F1 scores of 0.9621 and 0.8999 (GroupKFold LGB) / 0.9605 and 0.8958 (user-median LGB). These are the easiest classes, likely corresponding to rest/still and a common everyday activity with high inter-user consistency.
- **Class 4** (142 samples): Despite being the smallest class, it achieves F1 = 0.8714 (GroupKFold LGB) / 0.8817 (user-median LGB). Its distinctive sensor signature (presumably a highly specific posture or vigorous activity) makes it relatively easy to identify even with few examples.
- **Class 3** (656 samples): F1 = 0.6863 (GroupKFold LGB) / 0.6985 (user-median LGB). Confused with other classes frequently, though not catastrophically.

- **Class 5** (526 samples): F1 = 0.6745 (GroupKFold LGB) / 0.6600 (user-median LGB), with recall of only 0.6027 / 0.5627 respectively. The model frequently misses true class-5 instances, suggesting high inter-user variability in how this activity manifests in 1-second summary statistics.
- **Class 2** (358 samples): F1 = 0.1799 (GroupKFold LGB) / 0.2269 (user-median LGB), by far the worst. Precision 0.3583 / 0.3642 but recall only 0.1201 / 0.1648 — most true class-2 instances are classified as something else. No model variant solved this problem because the training signal is too weak and between-user variance too high for this class. The temperature scaling analysis confirmed this: class 2 had the highest temperature parameter ($T = 1.131$), meaning the model is overconfident on the few class-2 predictions it does make, yet still misses the vast majority.

Optimal temperatures per class:

[1.072, 1.086, 1.131, 1.014, 0.932, 1.078]

OOF macro-F1 before calibration: 0.7133 (RF+LGB tabular blend OOF)

OOF macro-F1 after calibration: 0.7129

Class 4 ($T = 0.932$) was the only under-confident class — the model was actually too conservative on its clearest signal. Temperature calibration had essentially no effect on label assignments (OOF F1 changed by only -0.0004), because the model's probability ordering was already well-ranked and the temperatures were all close to 1.0.

This per-class difficulty distribution means that virtually all macro-F1 score differences between submissions are driven by how well each method handles classes 2, 3, and 5 — not the majority classes.

Overall Submission Progression

The project evolved through three distinct phases, summarized below to provide context for answering question 2 – 4.

Phase 1 — Early exploration focused on establishing a working baseline and testing whether adding temporal or frequency-domain information to a simple Random Forest could improve activity recognition. The core question was whether the sequential structure of the 300-second recordings contained signal that flat global statistics discarded, and whether standard frequency features like FFT energy were a useful way to capture it.

Phase 2 — Architecture and ensemble upgrades shifted focus to replacing the shallow CNN-LSTM with a deeper residual sequence model, expanding the hand-crafted feature set substantially, and combining multiple tabular learners (Random Forest and LightGBM) with the CNN through soft-voting ensembles. This phase also introduced proper cross-validation and explored whether additional models (XGBoost, augmentation, more CNN seeds, higher CNN ensemble weight) could push performance further.

Phase 3 — Private generalization experiments / Generalization to unseen users was driven by the observation that strong public leaderboard scores did not reliably transfer to the private test set, where all users were completely unseen during training. Experiments focused on training and evaluating under GroupKFold by user identity, reducing the CNN's weight in the ensemble to limit user memorization, applying temperature calibration to the tabular models' probability outputs, and exploring user-level feature normalization to reduce sensor baseline drift across people.

Section 2: Preprocessing Techniques and Performance

Impact

The following preprocessing techniques were applied across versions :

2.1 StandardScaler Normalization (used in all versions)

All tabular features fed to Random Forest and LightGBM were standardized with StandardScaler (zero mean, unit variance) computed on training data and applied to test data. This is essential for LightGBM's regularization parameters (reg_alpha, reg_lambda) to function uniformly across features with very different scales (e.g., FFT spectral energy values in the thousands vs. autocorrelation values in [-1, 1]).

For the CNN, a per-feature axis normalization was applied directly to the 300×6 sequence arrays:

```
mu      = X_train_seq.mean(axis=(0, 1), keepdims=True)
sigma   = X_train_seq.std(axis=(0, 1), keepdims=True) + 1e-7
X_train_seq = (X_train_seq - mu) / sigma
```

This normalizes each of the 6 channels independently across all timesteps and samples. The 1e-7 prevents division by zero. Per-channel normalization is critical because mean_x/y/z values center around gravity (~1g) while std_x/y/z values are typically an order of magnitude smaller; treating them on the same scale would cause the network to ignore the std channels.

2.2 Constant-Length Padding

Files with fewer than 300 rows were zero-padded:

```
if len(sig) < 300:
    sig = np.pad(sig, ((0, 300 - len(sig)), (0, 0)), mode='constant')
```

Zero-padding at the tail preserves the actual signal at the head, which is appropriate since activity is continuous through the recording window. This is preferable to repeating-edge or symmetric padding, which would artificially inflate autocorrelation at lag 1.

Impact: The vast majority of files are exactly 300 rows long, so this rarely fires in practice, but its absence would cause shape errors in the batch-stacked CNN input array.

2.3 User-Median Normalization (private experiments)

In the private generalization experiments, user-level median normalization was introduced to reduce inter-user sensor baseline drift caused by different wearing angles:

```
baseline = user_median_dict[c] # computed from all of user's
training files
add_series_stats(feats, f"{c}_median_norm", v - baseline)
```

The idea is that each user's median accelerometer reading reflects their typical wrist posture during rest, and subtracting it makes features more invariant to between-user placement differences.

Impact: This produced `submission_private_user_median_lgb` with a public score of 0.7557, lower than the GroupKFold LGB-only baseline of 0.7717. The likely reason is a train/test mismatch: training files use user-specific medians computed from known user identities, but test files fall back to a global median across all training users, creating a systematic feature distribution shift at inference time that did not exist during training. The technique has sound motivation but requires knowing test user identities to work correctly. The per-fold OOF variation was high (0.6415 in fold 1 vs. 0.7520 in fold 2), confirming that some user groups are genuinely harder to generalize across, which median normalization alone cannot resolve.

2.4 Temperature Scaling (private experiments)

Per-class temperature scaling was applied to calibrate the tabular ensemble's probability outputs:

```
scaled_test = log_test / T_opt # T_opt learned from OOF via NLL
minimization
```

Optimal temperatures were [1.072, 1.086, 1.131, 1.014, 0.932, 1.078], indicating mild overconfidence across most classes ($T > 1$ softens probabilities). Class 4 ($T = 0.932$) was the only under-confident class. Class 2 had the highest temperature ($T = 1.131$), confirming that when the model does predict class 2, it is overconfident despite also missing most true class-2 instances.

Impact: The RF+LGB tabular blend OOF macro-F1 went from 0.7133 to 0.7129 after calibration — essentially no change. Public score was 0.7655, comparable to un-calibrated tabular blends. Calibration matters when probabilities are far from optimal (e.g., very overconfident predictions get the argmax wrong). Here the tabular model's probability ordering was already well-calibrated enough that rescaling temperatures near 1.0 did not change label assignments.

2.5 Data Augmentation (submission_v2)

`submission_v2` added two augmentation strategies to the CNN training: random time-scaling (stretching/compressing the 300-timestep sequence) and two CNN seeds averaged. The public score was 0.7676, below `v3`'s 0.7770 which used a single CNN without augmentation.

This is a counterintuitive result. Standard time-series augmentation advice suggests that cropping, scaling, and jittering should improve generalization. For wrist accelerometer HAR data, however, the activity window is already a fixed 5-minute segment, and each row represents a 1-second summary of mean/std sensor readings — not a raw sample. Stretching or compressing these summary statistics does not correspond to any realistic physical variation in wrist motion. The augmentation generates synthetic data that does not match the real data distribution, making the CNN learn invariances that are actually spurious for this representation. This explains why v2 was weaker than v3 despite the extra complexity. Additionally, v2's 0.7676 rerun dropped to 0.7526 (−0.0150), while v3's 0.7770 rerun dropped to only 0.7654 (−0.0116), confirming that augmentation increased rather than reduced variance.

2.6 Feature Engineering as Preprocessing

The richest preprocessing step was feature construction. Moving from 26 basic features to ~60 features (and later 268 in private experiments) was the single largest performance driver across the entire project.

Feature Group	Columns Added	Addresses
Global time-domain stats (mean, std, min, max)	24	Basic signal summary per axis
Extended stats: range, median, IQR, skew, kurtosis, energy, RMS, ZCR, p10/p90	30	Distribution shape, extremes
Autocorrelation lags 1, 5, 10, 30	12	Periodic motion rhythm
Temporal segments (early/mid/late thirds, then 5 segments)	18 – 30	Within-recording dynamics
FFT: peak frequency, spectral energy, entropy, top-3 components	20	Frequency-domain rhythm
Cross-axis: resultant magnitude stats, SMA	8	Orientation-invariant intensity
Pairwise axis correlations (xy, xz, yz), tilt angles	5	Wrist orientation
Jerk magnitude (diff of mag)	6	Rate of change

Feature Set Statistics by Method

Method	Shape	Top Variance Feature	Top Corr Pair	lrl
Baseline RF	11020×30	global_min_mean_z (0.357)	global_mean_mean_x / rolling_mean_mean_x	0.9999
RF + LGB + Residual CNN (best)	11020×69	fft_y_spectral_energy (1.94e7)	mag_mean / mag_energy	0.9895
LGB + XGB + multi-seed CNN (extended)	11020×133	fft_y_spectral_energy (1.94e7)	fft_x_peak_amp / fft_x_top1_amp	0.9902
Person- independent LGB (normalised)	11020×112	n_x_fft_spectral_energy (3841.4)	n_z_kurt / m_z_kurt	1.0000

All feature matrices were free of NaN and Inf values across every method. The baseline used 30 global time-domain statistics; the core rich feature set expanded this to 69 features by adding FFT spectral energy, autocorrelation at lags 1/5/10/30, IQR, skew, kurtosis, ZCR, temporal segment statistics, cross-axis correlations, and tilt angles. The extended set further grew to 133 features with 5-segment magnitude statistics and jerk features. The near-perfect correlation between `global_mean_mean_x` and `rolling_mean_mean_x` ($lrl = 0.9999$) in the baseline indicates redundant rolling-mean features that were superseded in later versions.

The most impactful preprocessing step was `StandardScaler` normalization applied to all tabular features, ensuring `LightGBM`'s regularization parameters (`reg_alpha`, `reg_lambda`) functioned uniformly across features with very different scales (e.g., FFT spectral energy in the thousands versus autocorrelation values in $[-1, 1]$). For the CNN, per-channel normalization was applied across all timesteps and samples independently, which was critical because `mean_x/y/z` values center around gravity (~ 1 g) while `std_x/y/z` values are typically an order of magnitude smaller. Zero-padding was applied to files shorter than 300 rows, preserving the actual signal at the head of the sequence.

User-median normalization (subtracting each user's median per channel from their features) was tested in private experiments to reduce inter-user sensor baseline drift. It produced a public score of 0.7557, lower than the GroupKFold LGB baseline of 0.7717, due to a train/test mismatch: training used user-specific medians, while test files fell back

to a global median, creating a systematic feature distribution shift at inference time. Data augmentation (random time-scaling of the 300-timestep sequences) was tested in submission_v2 and produced a public score of 0.7676, below the un-augmented equivalent (0.7770). Stretching or compressing rows of pre-aggregated mean/std statistics does not correspond to any realistic physical variation in wrist accelerometer data, so the augmentation generated out-of-distribution synthetic samples that degraded generalisation.

Performance impact by feature group:

- Basic stats only (submission-2) $\rightarrow$ 0.7209
- Adding FFT energy/max/mean to RF+CNN-LSTM (submission-5) $\rightarrow$ 0.7227, a regression of -0.0310 . FFT summaries alone were too noisy without richer statistics to contextualize them, and they may have introduced confusion into the RF component.
- SMA + peak frequency + spectral entropy replacing full FFT energy (submission_SMA) $\rightarrow$ 0.7475. Focusing on dominant rhythm rather than total spectral energy gave a more stable signal.
- Full rich feature set — autocorr lags, IQR, skew, kurtosis, ZCR, p10/p90, segment stats, top-3 FFT components, cross-axis correlations, tilt angles — combined with RF + LGB + Residual CNN (submission_improved) $\rightarrow$ 0.7881, a gain of $+0.0344$ over submission-3 and the largest single improvement in the project.
- Extended to 268 features with 5-segment magnitude stats and jerk features (private experiments) $\rightarrow$ GroupKFold LGB-only OOF 0.7123, reflecting a more honest cross-user evaluation rather than a regression in capability.

The table below summarises the performance impact of each feature engineering step:

Configuration	Public Score	Delta
Baseline RF, 30 features	0.7209	—
RF + CNN-LSTM, same features	0.7537	+0.0328
RF + CNN-LSTM + FFT energy/max/mean	0.7227	-0.0310
RF + CNN-LSTM + SMA + peak frequency + spectral entropy	0.7475	+0.0248
RF + LGB + Residual 1D-CNN, full rich feature set	0.7881	+0.0406

Adding raw FFT energy/max/mean to the CNN-LSTM baseline degraded performance by -0.0310 . These coarse spectral summaries are dominated by low-frequency components, sensitive to baseline drift between users, and partially redundant with what the CNN already captures from the raw sequence. Switching to SMA + peak frequency + spectral entropy recovered $+0.0248$ because peak frequency (the dominant repetition rate) and spectral entropy (concentration vs. diffuseness of the spectrum) are more user-invariant. The largest single gain ($+0.0406$) came from the complete pipeline rebuild combining a richer feature set, LightGBM alongside Random Forest, and a Residual 1D-CNN replacing the CNN-LSTM.

Section 3: Aligning Activity Labels with Sequential Accelerometer

Readings

The Alignment Problem

Each file's label applies to the entire 300-second recording — there is no frame-level annotation. The challenge is therefore to extract representations from the sequential readings that are informative for the global activity label. Two parallel approaches were used: global statistical feature extraction (for the tabular models) and direct sequence modeling (for the CNN), combined via soft-voting ensemble.

Approach A: Global Statistical Features (RF and LightGBM)

The tabular models do not process the sequence directly. Instead, temporal dependencies are captured by summarizing the 300-row sequence into a fixed-length feature vector. The key temporal features are:

Autocorrelation at multiple lags. For each axis ax and lags $[1, 5, 10, 30]$:

```
feats[f'm_{ax}_autocorr_lag{lag}'] = pd.Series(m).autocorr(lag=lag)
```

Autocorrelation at lag 1 captures second-to-second smoothness (high for slow movements like standing, low for rapid activities). Lag 10 and lag 30 capture medium-term

periodicity: walking has a step cycle of $\sim 1 - 2$ seconds, so autocorrelation at lags 1 – 2 will be high. Stair climbing has a different rhythm. This is how periodic motion rhythm is encoded as a tabular feature.

FFT dominant frequencies. For each axis, the FFT peak frequency and top-3 component frequencies were extracted:

```
fft_vals = np.abs(fft(m))[1:len(m) // 2]
feats[f'fft_{ax}_peak_freq'] = int(np.argmax(fft_vals)) + 1
feats[f'fft_{ax}_spectral_energy'] = float((fft_vals ** 2).sum())
feats[f'fft_{ax}_spectral_entropy'] = float(stats.entropy(fft_safe /
fft_safe.sum()))
```

The dominant frequency directly encodes the repetition rate of periodic activity. Walking at ~ 1.5 steps/second will produce a peak around frequency index 7 – 8 (given a 300-second window at 1 Hz sampling). Static activities produce low spectral energy and high spectral entropy (flat, noise-like spectrum).

Temporal segment statistics. The 300-row sequence was divided into 3 (and later 5) equal segments, and mean/std statistics were computed per segment:

```
thirds = np.array_split(m, 3)
for t_idx, seg in enumerate(thirds):
    feats[f'm_{ax}_seg{t_idx}_mean'] = seg.mean()
    feats[f'm_{ax}_seg{t_idx}_std'] = seg.std()
```

This captures non-stationarity within the recording: if an activity changes partway through the 5-minute window (e.g., someone starts moving after being stationary), the difference between early and late segment means will be large. Activities with consistent behavior will have similar statistics across all segments.

Cross-axis correlation. Pairwise Pearson correlations between axes (xy, xz, yz) were computed. Different activities produce characteristic coupling patterns between axes: walking couples vertical (z) and horizontal (x) motion in a specific phase relationship, while stair climbing produces a different coupling due to the elevation component.

Tilt angle features. The arctan2 of the mean acceleration components was computed to capture wrist orientation:


```
feats['tilt_xy'] = float(np.arctan2(mean_x_g, np.sqrt(mean_y_g**2 +
mean_z_g**2)))
```

Static activities like standing or sitting produce consistent tilt angles determined by the user's rest wrist posture, while dynamic activities produce variable tilt that averages to a different value.

Approach B: Direct Sequence Modeling (CNN architectures)

The CNN processes the full 300×6 sequence directly, learning temporal representations end-to-end. Two architectures were used across submissions:

CNN-LSTM (submissions 2 – SMA): The 300-row sequence was reshaped into 10 windows of 30 timesteps each. A TimeDistributed Conv1D(64, kernel=3) + BatchNorm + GlobalAveragePooling extracted local features within each 30-second window, and an LSTM(64, dropout=0.2) captured dependencies across the 10 windows. This architecture captures temporal progression at a coarse scale only (10 points of context). It was trained for a fixed 20 epochs with no early stopping and a single seed, making it high-variance due to random weight initialisation with no held-out stopping criterion.

Residual 1D-CNN (submission_improved onward): The flat 300×6 sequence was processed by a residual CNN with a Conv1D(64, kernel=7) stem followed by three residual blocks with filters [64, 128, 128], each using two Conv1D(5) layers with BatchNorm, residual skip connections, MaxPooling(2), and Dropout(0.2). In the private experiments, filters were expanded to [128, 256, 256] with stronger Dropout (0.25, 0.4, 0.3). Residual skip connections allow the network to combine low-level (raw motion amplitude) and high-level (multi-second pattern) features at all depths:

```
# Residual block (simplified)
res = layers.Conv1D(filters, 1, padding='same')(x) # skip
connection
x = layers.Conv1D(filters, 5, padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv1D(filters, 5, padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Add()(x, res) # residual
merge
x = layers.MaxPooling1D(2)(x)
```

At the end, multi-scale global pooling concatenates both average and max-pool representations:

```
avg_pool = layers.GlobalAveragePooling1D()(x) # average activity level
max_pool = layers.GlobalMaxPooling1D()(x) # peak activity moments
x = layers.Concatenate()([avg_pool, max_pool])
```

This is the key alignment mechanism for the CNN: GlobalAveragePooling aggregates the activity signature across all 300 timesteps into a single fixed-length vector. The average pool captures the typical temporal pattern; the max pool captures the most distinctive moment in the recording. Together they form the learned temporal summary that feeds the classification head.

Training used EarlyStopping (patience=12, restore_best_weights=True) and ReduceLROnPlateau (patience=5, factor=0.5). For the three low-weight CNN seeds in the private experiments, the logged best validation accuracies were 0.8966 (seed=0), 0.8875 (seed=42), and 0.8929 (seed=43).

v10 bigger CNN with label smoothing and spatial dropout: This was an attempt to add further regularization. Public scores were 0.7320 (seed=42) and 0.7469 (seed=0), both below any Phase 2 submission. Label smoothing replaces hard targets with soft distributions (0.9 for the true class), which discourages the confident predictions needed to correctly classify the minority classes (2, 5) that require sharp decision boundaries. This is why stronger regularization hurt rather than helped.

CNN Training Behaviour (R1 = first try; R2 = re-run)

Method	Run	Final train loss	Final val loss	Train/val gap	Final train acc	Final val acc	Epochs to converge	Overfit ?
Baseline RF	—	—	—	—	—	—	—	—
Hybrid RF +	R1	0.3394	— (no val)	—	0.8789	—	20/20	unknown

CNN-LSTM								
Hybrid RF + CNN-LSTM	R2	0.3432	— (no val)	—	0.8825	—	20/20	unknown
FFT + CNN-LSTM	R1	0.3374	— (no val)	—	0.8835	—	20/20	unknown
FFT + CNN-LSTM	R2	0.3414	— (no val)	—	0.8820	—	20/20	unknown
SMA + peak-frequency + CNN-LSTM	R1	0.3495	— (no val)	—	0.8799	—	25/25	unknown
SMA + peak-frequency + CNN-LSTM	R2	0.3529	— (no val)	—	0.8776	—	25/25	unknown
BEST RF + LGB + Residual CNN	R1	0.2256	0.3337	0.1081	0.9187	0.9038	34 (early stop, best ep.22)	Mild
BEST RF + LGB + Residual CNN	R2	0.1425	0.4037	0.2612	0.9465	0.8848	31 (early stop, best ep.19)	Yes
LGB + Residual CNN + augmented CNN (run 1)	R1	0.0057	0.0900	0.0843	0.9980	0.9746	60/60 (best ep.57)	Yes

LGB + Residual CNN + augmente d CNN (run 2)	R1	0.2069	0.244 1	0.0372	0.926 0	0.926 5	12 (early stop, best ep.1)	Minim al
LGB + Residual CNN + augmente d CNN (run 1)	R2	0.0060	0.124 8	0.1188	0.997 9	0.971 0	53 (early stop, best ep.41)	Yes
LGB + Residual CNN + augmente d CNN (run 2)	R2	0.2101	0.245 4	0.0353	0.925 1	0.925 1	12 (early stop, best ep.1)	Mild
LightGB M 5-fold + single CNN	R1	0.1220	0.424 7	0.3027	0.954 1	0.892 9	34 (early stop, best ep.22)	Yes
LightGB M 5-fold + single CNN	R2	0.1195	0.492 0	0.3725	0.958 9	0.875 7	26 (early stop, best ep.14)	Yes
RF + LGB + Residual CNN weight boosted	R1	0.1474	0.492 8	0.3454	0.943 7	0.895 6	26 (early stop, best ep.14)	Yes
RF + LGB + Residual CNN	R2	0.1443	0.509 2	0.3649	0.945 9	0.886 6	24 (early stop,	Yes

weight boosted							best ep.12)	
CNN seed search	R1	best seed=2: val_loss=0.3551, val_acc=0.8966						Mild
CNN seed search	R2	best seed=0: val_loss=0.3425, val_acc=0.8993					11 epochs shown	Mild
Person-independent LightGBM normalized features	R1/R2	0.2613 (ep.11)	0.4373	0.1760	0.9093	0.8893	11 (partial)	Mild
Bigger CNN + label smoothing + spatial dropout	R1	best seed=42: val_loss=0.3516, val_acc=0.9065						Mild
Bigger CNN + label smoothing + spatial dropout	R2	best seed=0: val_loss=0.2898, val_acc=0.9138						Mild

Approach C: Ensemble Alignment

The final prediction for submission_improved combines both approaches via soft-voting probability averaging:

```
final_preds = np.argmax(
    0.20 * rf_probs + 0.35 * lgb_probs + 0.45 * cnn_probs,
    axis=1
)
```

RF and LGB capture temporal dependencies through engineered statistics (autocorrelation, FFT, segment features). The CNN captures learned temporal patterns directly from the sequence. Their ensemble benefits from diversity: when a tabular model mis-classifies a borderline activity because its FFT features are ambiguous, the CNN's direct temporal pattern recognition may correctly identify it, and vice versa. The GroupKFold OOF analysis revealed that the optimal weights shift from RF=0.20, LGB=0.35, CNN=0.45 (for public leaderboard, submission_improved) to RF=0.25, LGB=0.50, CNN=0.25 (for submission_private_low_cnn_ensemble) and RF=0.20, LGB=0.50, CNN=0.30 (for the GroupKFold OOF weight sweep), because the tabular models' hand-crafted features are more robust to user-level distribution shift than the CNN's learned temporal patterns.

LightGBM 5-Fold OOF CV F1 (R1 = first try; R2 = re-run)

Method	Run	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	OOF mean
LGB + Residual CNN + augmented CNN ensemble	R1	0.7399	0.7100	0.7505	0.7410	0.7472	0.7380
LGB + Residual CNN + augmented CNN ensemble	R2	0.7300	0.7275	0.7101	0.7469	0.7307	0.7290
LightGBM 5-fold + single CNN	R1	0.7399	0.7100	0.7505	0.7410	0.7472	0.7380
LightGBM 5-fold + single CNN	R2	0.7300	0.7275	0.7101	0.7469	0.7307	0.7290
LGB + XGB + multi-seed CNN extended features (LGB)	R1	0.7548	0.7228	0.7551	0.7505	0.7737	0.7521

LGB + XGB + multi-seed CNN extended features (LGB)	R2	0.7537	0.7488	0.7171	0.7623	0.7636	0.7487
LGB + XGB + multi-seed CNN extended features (XGB)	R1	0.7425	0.7125	0.7388	0.7454	0.7574	0.7399
LGB + XGB + multi-seed CNN extended features (XGB)	R2	0.7559	0.7649	0.7108	0.7502	0.7402	0.7446
RF + LGB 5-fold + multi-seed CNN	R1	0.7336	0.7122	0.7416	0.7275	0.7432	0.7322
RF + LGB 5-fold + multi-seed CNN	R2	0.7335	0.7306	0.6984	0.7513	0.7366	0.7299
Person-independent LightGBM normalized features	R1	0.7380	0.7147	0.7387	0.7387	0.7492	0.7366
Person-independent LightGBM normalized features	R2	0.7515	0.7238	0.7020	0.7342	0.7443	0.7313

Section 4: Ablation Study

The ablation study tracks the contribution of individual design choices by examining score changes as components were added or modified across submissions. All public scores are from Kaggle's public leaderboard (macro-F1). OOF scores are from cross-validation on training data.

The project evolved through three distinct phases.

Phase 1 — Early exploration (submission-2 through submission_SMA): Starting from the RF-only baseline (0.7209), the first hybrid (submission-3, 0.7537) demonstrated the value of adding a CNN-LSTM. FFT features added on top of that (submission-5, 0.7227) unexpectedly dropped performance, and the SMA + peak-frequency variant (submission_SMA, 0.7475) only partially recovered. These early results revealed that

adding frequency features to a simple CNN-LSTM with no cross-validation was inconsistent, as the single-seed training introduced high run-to-run variance.

Phase 2 — Major architecture upgrade (submission_improved through submission_v6): submission_improved replaced the CNN-LSTM with a Residual 1D-CNN, added LightGBM alongside Random Forest, and introduced a substantially richer feature set, reaching the overall best public score of 0.7881 — a gain of +0.0344 over submission-3. Subsequent versions (v2 – v6) explored cross-validation, multi-seed averaging, XGBoost stacking, and CNN weight adjustments, none of which surpassed submission_improved on the public leaderboard.

Phase 3 — Private generalization experiments: After observing the private leaderboard drop (ranking ~44th when v2 and v6 were selected as final submissions), targeted experiments addressed user-distribution shift: GroupKFold by user, reduced CNN weight, temperature scaling, and user-median normalization. These scored 0.7557 – 0.7800 on the public leaderboard. submission_private_low_cnn_ensemble (0.7800) became the best among these, confirming that reducing CNN weight and applying GroupKFold training improves cross-user generalization.

4.1 Feature Engineering Ablation

Configuration	Public Score	Delta vs. previous row
Baseline: RF-only, 30 features (global stats + rolling mean + magnitude)	0.7209	—
RF + CNN-LSTM, same global-stat features (submission-3)	0.7537	+0.0328
RF + CNN-LSTM + FFT energy, max, mean per axis (submission-5)	0.7227	−0.0310
RF + CNN-LSTM + SMA + peak frequency + spectral entropy, replacing full FFT (submission_SMA)	0.7475	+0.0248
Full rebuild: RF + LGB + Residual 1D-CNN, complete rich feature set: autocorr lags, IQR, skew, kurtosis, ZCR, p10/p90, segment stats, top-3 FFT, cross-axis corr, tilt (submission_improved)	0.7881	+0.0406

Note: The last row reflects a complete pipeline rebuild (new architecture, new tabular model, new feature set), not an incremental addition to submission_SMA. The +0.0406 delta reflects the combined effect of all three changes.

Observation: Simply adding FFT energy features to the CNN-LSTM baseline degraded performance (-0.0310). There are several reasons for this. First, FFT energy/max/mean computed over the full 300-row window are coarse summaries that are at least partially redundant with what the CNN-LSTM already captures from the raw sequence — the CNN sees the raw signal and implicitly learns spectral characteristics, so adding redundant hand-crafted spectral features adds noise without adding independent discriminative signal. Second, total spectral energy is dominated by low-frequency components and is sensitive to baseline drift between users; it varies more between users doing the same activity than between activities within the same user, making it a noisy feature for a classifier trained without user-level normalization. Third, these three new features (energy, max, mean) are computed only on the tabular RF side, not integrated into the CNN. They expand the RF's feature space slightly while diluting the relative importance of the already-informative global stats, potentially disrupting the RF's existing decision boundaries without providing enough new signal to compensate.

Switching to SMA + peak frequency + spectral entropy (submission_SMA) recovered significantly (+0.0248 vs. submission-5). Peak frequency is semantically stable: it identifies the dominant repetition rate of the activity, which is consistent across users performing the same activity (e.g., walking always produces a dominant step frequency regardless of the user's arm length or wearing angle). Spectral entropy captures whether the spectrum is concentrated (periodic activity) or diffuse (random motion), which is also user-invariant. These features address a different dimension of the signal than global statistics, adding genuine complementary information rather than noisy redundancy.

The most important jump came from the complete rebuild in submission_improved (+0.0406). The gain comes from three simultaneous sources that cannot be individually isolated from this table alone: the feature enrichment (autocorrelation at multiple lags captures rhythm periodicity; IQR, skew, and kurtosis capture distributional shape beyond mean and variance; temporal segment stats capture within-recording non-stationarity; cross-axis correlations capture motion geometry), the addition of LightGBM (which uses gradient boosting and handles feature interactions differently from RF's bagging approach, reducing correlated errors), and the switch to Residual 1D-CNN (which processes the full 300-step sequence end-to-end rather than 10 coarse windows). All three changes compound.

4.2 Model Architecture Ablation

Architecture	Public Score	Delta vs. previous row
RF only, global-stat features (submission-2)	0.7209	—
RF + CNN-LSTM, global-stat features (submission-3)	0.7537	+0.0328
LGB 5-fold + single Residual 1D-CNN, rich features (v3)	0.7770	+0.0233
RF + LGB + Residual 1D-CNN, full rich features (submission_improved)	0.7881	+0.0111
LGB + XGB (minor) + 3-seed CNN (v4)	0.7547	−0.0223 vs. v3
Bigger CNN + label smoothing + spatial dropout (v10, seed42)	0.7320	−0.0227 vs. v4

Observation: The +0.0233 gain from submission-3 to v3 reflects two simultaneous changes that cannot be cleanly separated: the switch from CNN-LSTM to Residual 1D-CNN, and the addition of LightGBM alongside RF. The architectural change likely accounts for the majority of the gain. The CNN-LSTM segmented the 300-row sequence into 10 coarse 30-second windows before modeling temporal dependencies, meaning any pattern shorter than ~30 seconds was averaged away before the LSTM saw it. The Residual 1D-CNN processes the full flat 300-timestep sequence with convolutional kernels of width 5 – 7, giving it access to all temporal scales simultaneously. Residual skip connections allow gradients to flow directly to early layers, preventing the vanishing gradient problem that limits plain CNNs on long sequences. Multi-scale global pooling (average + max concatenated) captures both the typical activity signature across the full recording and the most distinctive moment — the max pool is particularly useful for activities with brief but characteristic motion bursts that would be diluted by averaging. Adding LightGBM alongside RF diversifies the tabular component because LGB uses gradient boosting (sequential trees correcting residuals) while RF uses bagging (parallel trees averaging), so they make systematically different errors on boundary cases.

Adding XGBoost in v4 did not improve performance (0.7547 vs. 0.7770 in v3, a drop of −0.0223). The optimal ensemble blend found by the OOF weight search was LGB=0.40, XGB=0.10, CNN=0.50 — XGB received only a 0.10 weight, indicating it contributed almost nothing beyond what LGB already captured. Both are gradient-boosted decision trees trained on the same 133-feature set, so their predictions are highly correlated: when LGB misclassifies a borderline instance because certain features are ambiguous for that user, XGB trained on the same features will almost certainly make the same error. The

OOF ensemble proxy score of 0.7494 looked promising, but this was optimistic because the CNN component — which received the highest weight of 0.50 — was not evaluated in a proper cross-validated OOF structure, inflating the apparent benefit of high CNN weighting. The public score of 0.7547 revealed the true cost: a CNN-dominant ensemble without proper cross-validation on the CNN component overfits to user-specific patterns not present in the test set.

The bigger CNN with label smoothing and spatial dropout (v10) degraded performance by -0.0227 vs. v4. Label smoothing redistributes probability mass away from the true class (e.g., 0.9 for the correct label instead of 1.0), which is harmful here because the minority classes (2, 5) are already underrepresented and require sharp decision boundaries — softening the targets makes the model treat a correct minority-class prediction as nearly as good as a wrong one, degrading recall on exactly the classes that dominate macro-F1. Spatial dropout drops entire convolutional feature map channels during training. For majority classes with thousands of examples this acts as useful regularization, but for class 4 (142 samples) and class 2 (358 samples) losing entire feature channels in each forward pass prevents the network from reliably learning the sparse signatures these classes produce, effectively amplifying the sample scarcity problem.

4.3 Cross-Validation Strategy Ablation

CV Strategy	OOF F1-macro	Public Score	Notes
No CV (single train/val split, ~90/10)	N/A	0.7537 (submission-3)	Single seed, fixed 20 epochs
StratifiedKFold 5-fold, LGB-only OOF (v3)	0.7380	0.7770	LGB-only OOF; +0.0233 vs. no CV
StratifiedKFold 5-fold, ensemble OOF (v4)	0.7494	0.7547	Best-blend OOF (LGB=0.40, XGB=0.10, CNN=0.50); LGB-only OOF was 0.7521
GroupKFold 5-fold by user (private LGB only)	0.7123	0.7717	More honest, stable cross-user
GroupKFold 5-fold by user (private GKF ensemble)	0.7167	0.7744	RF+LGB+CNN ensemble under GroupKFold

Observation: The OOF column is not directly comparable across v3 and v4 — v3 reports LGB-only OOF (0.7380) while v4 reports the best ensemble blend OOF (0.7494, at LGB=0.40, XGB=0.10, CNN=0.50). v4's LGB-only OOF was actually 0.7521, higher than v3's 0.7380, because v4 used a richer 133-feature set. XGB-only OOF was 0.7399. Despite the higher OOF, v4's public score (0.7547) was substantially lower than v3 (0.7770), a drop of -0.0223 .

There are several reasons why a higher ensemble OOF did not translate to a better public score. First, the CNN component (weight=0.50 in the optimal blend) was trained without cross-validation on the OOF search — the CNN predictions used in the weight sweep were in-sample or from a single fixed split, making the OOF proxy optimistic for any blend that up-weights the CNN. Second, the three CNN seeds (42, 7, 123) were averaged but not evaluated in a proper held-out fold structure, so the 0.7494 OOF proxy inflated the apparent benefit of high CNN weight. Third, adding XGBoost provided minimal genuine diversity over LGB (both are gradient-boosted trees on the same 133 features; XGB OOF 0.7399 vs. LGB OOF 0.7521 shows XGB was simply the weaker model), meaning the ensemble mostly averaged a stronger model with a weaker correlated one rather than combining complementary strengths.

StratifiedKFold still substantially outperforms no-CV for the LGB component (0.7521 vs. the implied single-split performance of submission-3's RF). The fundamental limitation of StratifiedKFold — shuffling files across users so the model sees some recordings from every user during training — applies equally to v3 and v4, which is why both produce optimistically high OOF scores relative to GroupKFold's 0.7123.

4.4 Ensemble Weight Ablation

Configuration	CNN Weight	OOF / Public Score	Notes
RF+LGB tabular blend only (GKF, best sweep)	0.00	OOF 0.7133	w_rf=0.35, w_lgb=0.65; no public submission
LGB only, GroupKFold	0.00	OOF 0.7123 / public 0.7717	Tabular-only public baseline
Private low-CNN ensemble	0.25	public 0.7800	Best private-aware submission; weights RF=0.25, LGB=0.50, CNN=0.25

submission_improved	0.45	public 0.7881	Best public score overall
submission_v6	0.55	public 0.7664	−0.0217 vs. CNN=0.45

Observation: The optimal CNN weight was 0.45 for the public leaderboard, but reducing it to 0.25 (used in submission_private_low_cnn_ensemble) was better for private generalization. The GroupKFold OOF weight sweep separately found an optimal 3-way split at RF=0.20, LGB=0.50, CNN=0.30 (OOF 0.7167), a closely related configuration. As CNN weight increases beyond 0.45, user-specific temporal patterns that the CNN memorized during training dominate the ensemble, degrading cross-user robustness. The tabular models (RF, LGB) rely on hand-crafted features with built-in invariances (autocorrelation, spectral entropy, cross-axis correlations), making them more robust to user-level distribution shift. The optimal ensemble weight is therefore a tradeoff between exploiting the CNN's temporal pattern-matching strength and limiting its user-overfitting tendency.

4.5 Training Stability Ablation (Multi-Seed and Augmentation)

All Phase 2 submissions were re-run under identical code to measure run-to-run variance. submission_seed2 (seed=2 variant of the submission_improved architecture) and submission_v10_seed0 (seed=0 variant of v10_seed42) provide additional seed comparison points. The full rerun record is:

Submission	Original Score	Rerun Score	Delta	Key Factor
submission_improved	0.7881	0.7614 (submission_improved-2)	−0.0267	Largest drop; single seed, no CV averaging
submission_v2	0.7676	0.7526 (submission_v2-2)	−0.0150	Augmentation increased variance
submission_v3	0.7770	0.7654 (submission_v3-2)	−0.0116	Most stable Phase 2 method
submission_v4	0.7547	0.7678 (submission_v4-2)	+0.0131	Variance can go either direction

submission_v5	0.7484	0.7342 (submission_v5-2)	−0.014 2	Multi-seed CNN did not reduce variance
submission_v6	0.7664	0.7475 (submission_v6-2)	−0.018 9	High CNN weight amplified instability
submission_v10_seed 42	0.7320	—	—	Seed search only; seed=0 variant below
submission_v10_seed 0	0.7469	0.7521 (submission_seed0-2)	+0.005 2	Small positive variance
submission_seed2	0.7626	—	—	Seed=2 variant of submission_improved architecture

Observation: Score variability across reruns is not a simple downward trend — v4 improved from 0.7547 to 0.7678 on rerun, and v10_seed0 improved slightly from 0.7469 to 0.7521, showing that stochastic variation can go either direction. The sources of non-determinism are: GPU-level cuDNN nondeterminism (not eliminated by Python/TF seeds without TF_DETERMINISTIC_OPS=1), mini-batch ordering (shuffle=True in Keras fit), and Keras's random 10% validation split (no fixed seed), which changes which weights get selected by EarlyStopping.

The most striking rerun result is submission_improved, which dropped −0.0267 on rerun — the largest variance of any submission. Despite having the highest single-run public score (0.7881), this makes it the least reliable indicator of true model quality. The three seed variants of the submission_improved architecture — seed=0 (original, 0.7881), seed=2 (submission_seed2, 0.7626), and the rerun (submission_improved-2, 0.7614) — span a range of 0.0267, indicating the headline score was likely a fortunate seed draw. In contrast, v3 dropped only −0.0116, confirming that fewer CNN degrees of freedom (lower CNN weight, single seed, no augmentation) produces more reproducible results.

The augmentation applied in v2 (time-scaling of the 1-second summary rows) was conceptually flawed: stretching or compressing rows of pre-aggregated mean/std statistics does not correspond to any realistic physical variation in wrist accelerometer data, so it introduced distributional noise that made the CNN less discriminative while simultaneously increasing run-to-run variance (−0.0150 drop, vs. −0.0116 for un-augmented v3).

Based on the full rerun data, the observed score variation across independent runs ranges from -0.0267 to $+0.0131$, meaning models that differ by less than $\sim 0.010 - 0.015$ on a single evaluation cannot be reliably ranked. The private GKF ensemble (submission_private_gkf_ensemble, 0.7744) was trained with 5 GroupKFold folds, each using a single CNN seed, with fold predictions averaged at test time — this reduces fold-level variance but does not sweep multiple seeds per fold. It was not re-run separately, but its GroupKFold structure makes it inherently more stable than single-split methods.

4.6 Public-Private Leaderboard Gap Analysis

The private leaderboard revealed an important generalization failure. When v2 and v6 were selected as final private submissions, the private ranking dropped to approximately 44th place — below competitors whose public scores were as low as 0.7448, meaning models with substantially lower public scores generalized better to the private test set. By contrast, when submission_improved and submission_v3 were selected as final submissions in the prior week, the private ranking was 23rd. This reversal illustrates that public leaderboard score is not a reliable proxy for private generalization when the test distribution includes entirely new users.

Their rerun scores (v2: 0.7526, v6: 0.7475) indicate both were near the lower end of their true performance distributions, compounding the generalization penalty.

Submission	Public Score	Rerun Score	Key Risk Factor
submission_improved	0.7881	0.7614 (-0.0267)	Largest seed variance; CNN weight 0.45
submission_v3	0.7770	0.7654 (-0.0116)	Most stable; lower CNN weight, no augmentation
submission_v2	0.7676	0.7526 (-0.0150)	Unrealistic augmentation increased variance
submission_v6	0.7664	0.7475 (-0.0189)	CNN weight 0.55 amplified overfitting

The root cause of the private drop is user distribution shift. The 60 training users and the public/private test users are completely separate people. A CNN trained with a high weight in the ensemble will learn user-specific temporal patterns — the exact cadence, wrist tilt trajectory, and transition timing that characterize individual people — in addition to activity-class patterns. These user-specific patterns do not transfer to new users and

actively hurt predictions when the test person's motion style differs from all 60 training users. v2's unrealistic augmentation (stretching and compressing rows of pre-aggregated mean/std statistics) distorted the CNN's learned feature distribution in a direction that does not correspond to any real physical variation, adding noise that degraded generalization without improving robustness. v6's CNN weight of 0.55 gave the most user-overfitting component the most influence over the final prediction. A better selection criterion would have been: prefer models with lower CNN weight, trained under GroupKFold by user, confirmed stable across at least two independent reruns — all three conditions that `submission_private_low_cnn_ensemble` satisfied.

Summary of All Submissions

Submission	Key Design	Public Score	Key Observation
submission-2	RF-only, 30 global-stat features	0.7209	Naive baseline; statistics alone insufficient
submission-3	RF + CNN-LSTM, 26-col global stats	0.7537	+0.0328 from adding temporal sequence model
submission-5	RF + CNN-LSTM + FFT energy/max/mean	0.7227	FFT energy features hurt RF; -0.0310 regression
submission_SMA	RF + CNN-LSTM + SMA + peak freq	0.7475	Peak freq more stable than full FFT energy
submission_improved	RF + LGB + Res-CNN, full rich features	0.7881	Best public score; rerun dropped to 0.7614 (-0.0267)

submission_v3	LGB 5-fold + single Res-CNN	0.777 0	Most stable Phase 2 method; rerun -0.0116
submission_v2	CNN aug (time-scale) + 2 seeds	0.767 6	Augmentation flawed; rerun dropped to 0.7526
submission_v4	LGB + XGB + 3-seed CNN	0.754 7	XGB correlated with LGB; rerun improved to 0.7678
submission_v5	RF + LGB 5-fold + multi-seed CNN (RF=0.15, LGB=0.30, CNN=0.55)	0.748 4	Rerun dropped to 0.7342; CNN weight too high
submission_v6	CNN weight 0.45 → 0.55	0.766 4	More CNN weight amplified user overfitting; rerun 0.7475
submission_v10_seed42	Bigger CNN + label smooth + spatial dropout	0.732 0	Over-regularized; label smoothing hurt minority classes
submission_v10_seed0	Same architecture, seed=0	0.746 9	+0.0149 vs. seed=42; seed variance alone spans 0.0149
submission_private_low_cnn_ensemble	GroupKFold + RF/LGB/CNN=0.25/0.50/0.25	0.780 0	Best private-aware submission; lower CNN weight helps

submission_private_gkf_ensemble	GroupKFold 5 folds, single CNN seed per fold, averaged	0.774 4	GroupKFold structure reduces fold variance; GKF OOF CNN = 0.6772; GKF ensemble OOF = 0.7167
submission_private_lgb_groupkfold	GroupKFold LGB only, no CNN	0.771 7	Tabular-only competitive; GKF OOF = 0.7123
submission_private_lgb_xgb_stack	GroupKFold LGB + XGB blend	0.771 4	Best blend OOF 0.7140 (w_lgb=0.40, w_xgb=0.60)
submission_private_calibrated	Temperature scaling on RF+LGB tabular blend	0.765 5	Temperatures near 1.0; OOF F1 unchanged (0.7133→0.7129)
submission_private_user_median_lgb	User-median normalization + LGB	0.755 7	Train/test identity mismatch; OOF 0.7206
submission_seed2	submission_improved architecture, seed=2	0.762 6	−0.0255 vs. seed=0 original; confirms wide seed variance
submission_improved-2	submission_improved rerun (seed=0)	0.761 4	−0.0267 vs. original; largest rerun drop
submission_v2-2	submission_v2 rerun	0.752 6	−0.0150; augmentation increased

			variance confirmed
submission_v3-2	submission_v3 rerun	0.765 4	−0.0116; most stable Phase 2 method
submission_v4-2	submission_v4 rerun	0.767 8	+0.0131; stochastic variance can go either direction
submission_v5-2	submission_v5 rerun	0.734 2	−0.0142; multi-seed CNN did not reduce variance
submission_v6-2	submission_v6 rerun	0.747 5	−0.0189; high CNN weight amplified instability
submission_seed0-2	submission_v10_seed0 rerun	0.752 1	+0.0052 vs. original